

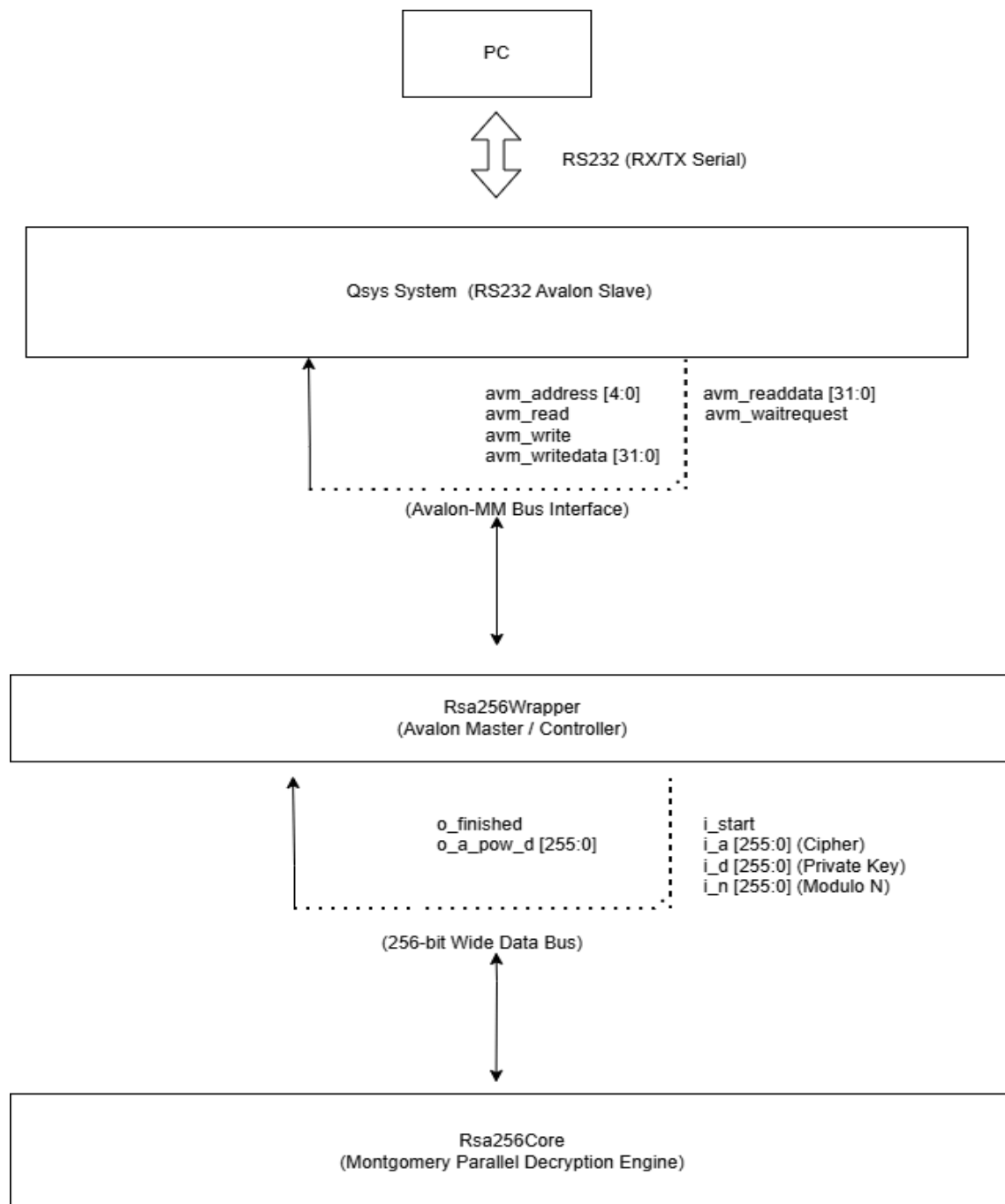
NTUEE DCLAB Lab 2: RSA256 解密機 結報

零、File Structure

```
team10_lab2
├── team10_lab2_report.pdf
├── src
│   ├── Rsa256Core.sv
│   ├── Rsa256Wrapper.sv
│   ├── DE2_115
│   │   ├── DE2_115.qsf
│   │   ├── DE2_115.sdc
│   │   ├── DE2_115.sv
│   │   └── DE2_115.sv.bak
│   ├── pc_python
│   │   ├── dec.bin
│   │   ├── enc.bin
│   │   ├── key.bin
│   │   ├── rs232.cpp
│   │   └── rs232.py
│   ├── golden
│   │   ├── dec1.txt
│   │   ├── dec2.txt
│   │   ├── dec3.txt
│   │   ├── enc1.bin
│   │   ├── enc2.bin
│   │   ├── enc3.bin
│   │   ├── key.bin
│   │   ├── key_ascii.txt
│   │   └── rsa.py
│   └── tb_verilog
│       ├── PipelineCtrl.v
│       ├── PipelineTb.v
│       ├── tb.sv
│       ├── test_wrapper.sv
│       ├── wrapper_input.txt
│       └── wrapper_output.txt
```

一、System Architecture 與 Data Path

本系統主要負責接收來自 PC 端的 RSA256 密文與金鑰，利用 FPGA 進行硬體解密加速後，將明文回傳至 PC 端。整體架構包含三大模組：Qsys System (SoC)、RSA256 Wrapper 以及 RSA256 Core。



● **Data Path 流程:**

1. **PC to FPGA:** PC 端透過 RS232 通訊協定, 將 32-byte 的 N、32-byte 的 d 以及 32-byte 的 cipher text 依序傳送給 FPGA。
2. **Qsys (RS232 IP):** Qsys 生成的 RS232 IP 作為 Avalon-MM Slave, 負責將序列訊號轉換為 8-bit 的 parallel data。
3. **RSA256 Wrapper:** 作為 Avalon-MM Master, 透過 Address Mapping (`BASE+0`, `BASE+4`, `BASE+8`) 輪詢 RX 與 TX 的狀態。Wrapper 會將接收到

的 8-bit 資料透過 Shift 組合拼接成 256-bit 的暫存器資料 (n_r , d_r , enc_r), 並在收集完畢後拉高 i_start 訊號交給 Core。

4. **RSA256 Core**: 接收到完整的 256-bit N 、 d 與 y 後, 開始執行 Montgomery 演算法解密。
5. **FPGA to PC**: Core 解密完成後拉高 $o_finished$ 。Wrapper 接收到 256-bit 明文後, 透過 Shift 的方式, 將最高的 31 Bytes 逐一寫入 TX_BASE 傳回 PC 端。

二、Hardware Scheduling (FSM & Algorithm Workflow)

硬體排程設計為了確保資料交握的正確性以及運算的高效性, 我們在 Wrapper 與 Core 皆設計了專屬的 FSM。

1. RSA256 Core FSM 與平行化架構

為了加速 RSA256 的解密流程, 我們將 Montgomery 演算法拆解為有限狀態機進行控制, 並實例化了兩個 **Montgomery** 運算子模組進行平行處理：

- **S_PREP**: 負責執行前置作業, 計算 $y * 2^{256}$ 。我們利用連續左移 1 bit 並判斷是否大於等於 N 進行減法, 共執行 256 次。
- **S_MONT**: 核心的 Montgomery 迴圈。在此狀態下, 我們同時啟動兩個硬體模組, 平行計算 $m * t^{2^{256} \bmod N}$ 以及 $t^2 * 2^{2^{256} \bmod N}$ 。透過平行化, 將原本需要極長週期的運算壓縮至約 6.5 萬個 Clock Cycles, 大幅提升硬體加速效率。

2. RSA256 Wrapper FSM

為了嚴謹處理 Avalon-MM 的交握協定, 並避免 Off-by-one 的資料錯位, Wrapper 的 FSM 被設計為「嚴格分離 Status 輪詢與 Data 讀寫」的兩階段架構：

- **S_GET_KEY / S_GET_DATA**: 先讀取 $STATUS_BASE$ 。只有當 RX_OK bit 為 1 時, FSM 才會轉移到 Read 狀態。
- 截斷填充 **Byte (0x00)**: 根據設計規範, PC 端為了避免溢位, 會在 31 Bytes 的明文前方填充一個 zero byte (0x00)。因此, Wrapper 在將 256-bit 明文回傳時, 會精準地從 [247:240] 開始截取資料, 並只循環傳送 31 次, 完美濾除填充的 0x00。

三、Bonus 實作: 連續解密多份密文 (Hot-Reload 機制)

本專案實作了簡報中要求的 Bonus 功能: 不需 **reset** 即可連續解密多份密文 (不同金鑰)。

- 實作原理: 在 Wrapper 的 **S_GET_DATA_STATUS** 狀態中, 我們加入了一個 **Idle Timeout Counter**。如果在等待下一份密文的過程中, 超過了 25,000,000 個 cycles (在 50MHz 時脈下約 0.5 秒) 都沒有收到 RX_OK 訊號, 硬體會判定 PC 端開啟了全新的傳輸週期 (可能更換了金鑰並重新執行 Python 腳本)。
- 效果: 觸發 Timeout 後, FSM 會自動重置回 **S_GET_KEY_STATUS** 狀態, 清空之前的金鑰暫存器, 重新準備接收新的 64-Byte N 與 d 。這賦予了系統強大的韌性與熱重載 (Hot-Reload) 能力, 使用者完全不需要去按板子上的 Reset 鍵 (**KEY0**)。

四、遇到的問題與解決辦法

1. VCS 模擬 Deadlock (Too slow, abort) 問題

- 問題描述: 初期在編寫 Core 時, 由於將巨大的迴圈邏輯寫在組合邏輯中, 或未接受完成訊號, 導致 FSM 卡在 S_MONT 狀態無窮迴圈, 觸發 Testbench 的超時強制中止。
- 解決辦法: 將 Montgomery 演算法拆解為獨立的 Sequential Logic 模組, 並透過計數器 (0~256) 控制迴圈。採用兩個 Montgomery 模組平行運算後, 成功在約 3.3M 個 cycles 內完成解密, 徹底解決超時問題。

2. Avalon-MM 讀取資料錯位 (Desync)

- 問題描述: 初期設計時, 在同一個 cycle 判斷 RX_OK 為 1 就立刻將 avm_readdata 存入金鑰暫存器, 導致讀入的是上一筆的 Status Code (如 0xC0), 使後續解密完全失敗。
- 解決辦法: 修改 Wrapper 的 FSM, 將「讀取 Status」與「讀取 Data」嚴格拆分為兩個不同的 State。確認 Status OK 後, 先給定 Data Address, 於下一個 cycle 才正式擷取資料, 確保資料對齊。

3. Python 端驗證失敗 (輸出位移)

- 問題描述: 解密計算正確, 但在比對輸出時, 發現硬體回傳的字串最前方多出一個 0x00, 導致後續明文字元全部往後平移 1 Byte。
- 解決辦法: 仔細查閱規格後, 發現明文僅有 31 Bytes, 且最高位元組為填充的 zero byte。將輸出匯流排的截取範圍從 [255:248] 修正為 [247:240], 即完美解決字串平移問題。

五、Fitter Summary & Timing Analyzer

Fitter Summary	
Fitter Status	Successful - Thu Mar 19 21:59:13 2026
Quartus II 64-Bit Version	15.0.0 Build 145 04/22/2015 SJ Full Version
Revision Name	DE2_115
Top-level Entity Name	DE2_115
Family	Cyclone IV E
Device	EP4CE115F29C7
Timing Models	Final
Total logic elements	6,014 / 114,480 (5 %)
Total combinational functions	5,467 / 114,480 (5 %)
Dedicated logic registers	2,723 / 114,480 (2 %)
Total registers	2723
Total pins	518 / 529 (98 %)
Total virtual pins	0
Total memory bits	0 / 3,981,312 (0 %)
Embedded Multiplier 9-bit elements	0 / 532 (0 %)
Total PLLs	1 / 4 (25 %)

Fitter Summary: 從 Fitter Summary 中可以看出本設計消耗的 Logic Elements 與 Registers 數量, 皆在 DE2-115 的 Cyclone IV 晶片資源容許範圍內。

Slow 1200mV 85C Model Fmax Summary				
	Fmax	Restricted Fmax	Clock Name	Note
1	32.56 MHz	32.56 MHz	my_qsys altpll_0 sd1 pll7 clk[0]	

Slow 1200mV 0C Model Fmax Summary				
	Fmax	Restricted Fmax	Clock Name	Note
1	36.13 MHz	36.13 MHz	my_qsys altpll_0 sd1 pll7 clk[0]	

Timing Analyzer: 系統的Fmax 滿足 Qsys PLL 所產生的系統時脈頻率需求, 無 Timing Violation (Setup/Hold time slacks 皆為正值), 確保硬體在下板後能夠穩定運作。

Unconstrained Paths			
	Property	Setup	Hold
1	Illegal Clocks	0	0
2	Unconstrained Clocks	0	0
3	Unconstrained Input Ports	0	0
4	Unconstrained Input Port Paths	0	0
5	Unconstrained Output Ports	1	1
6	Unconstrained Output Port Paths	1	1

六、心得與建議

這次 Lab 2 是我們第一次將大型演算法與 SoC 架構結合的挑戰。過程中我們深刻體會到軟體邏輯與硬體排程的巨大差異; 軟體中一個簡單的 for 迴圈, 在硬體裡卻需要謹慎設計 FSM 來避免組合邏輯過長或 Clock Cycle 浪費。

此外, 與 Avalon-MM 交握的除錯過程雖然痛苦, 但也讓我徹底理解了 Memory Mapped I/O 的底層通訊機制。從一開始看著 nWave 上的紅線發呆, 到最後精準抓出 0x00 padding 的 Bug 並加上 Timeout 機制, 非常有成就感。此外這次的經驗也告訴了我未來實作類似通訊介面時, 一定要將 Status Polling 與 Data Fetching 的狀態嚴格分離, 能少走許多冤枉路。